Projektuppgift

Introduktion till webbutveckling med HTML, CSS och JavaScript (DT224G)

Projektuppgift
Mamma's Bakery

Lucciana Ghazoul



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Lucciana Ghazoul, lugh2600@student.miun.se
Utbildningsprogram: TWEUG, Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

1. Sammanfattning

För det avslutande projektuppgift till kursen "Introduktion till webbutveckl-ing med HTML, CSS och JavaScript" har jag skapat webbsidan "Mamma's Bakery". Mamma's Bakery är en enkel och interaktiv webbplats som representerar ett litet bageri som användaren kan lätt beställa bakverk och lunch sallader med samma bra kvalitet som i stora företag genom att ringa eller använda kontaktformulär. Webbplatsen visar vad har bageriet för bakverk, tårtor, sallader och bröd med information som inkluderar ingredienser, pris, bilder på varan och en kort beskrivning. Webbplatsen är uppbyggd av fem huvudsidor som innehåller information som till exempel sortimentmeny, kontaktinformation samt presentation av företaget. Denna rapport kommer visa hela processen från idé till färdig webbplats. Webbplatsen är publicerad på GitHub och här är länken till den färdiga webbplatsen: https://lucciana-g.github.io/project_Lucciana-Ghazoul/

Nyckelord: JavaScript, responsiv design, Figma, HTML, CSS, GitHub, JSON.

2. Innehållsförteckning

1 Sammanfattning 2
2 Innehållsförteckning 3
3 Introduktion 4
3.1 Bakgrund och problemmotivering 5
3.2 Detaljerad problemformulering 5
4 Teori 5
5 Metod 6
6 Konstruktion 7
7 Resultat 8
8 Slutsatser 9

3. Introduktion

3.1. Bakgrund och problemmotivering

Nu för tiden om man vill köpa ett bakverk eller en tårta till exempel för en födelsedag vänder man sig till ett känt bageri i stan eller till en affär som säljer bakelser och tårtor av stora företag. I både fall kostar det mycket, det är ganska dyrt för en privat person eftersom mataffärer köpa för en viss kostnad från bagerier och sälja för dubbelt eller mer.

Här kommer Mamma's Bakerys roll eftersom Mamma's Bakery är ett litet bageri som har stort variant sortiment med samma bra kvalitet som stora företag och bagerier, men i rimliga priser och överkomliga för alla.
Idén till "Mamma's Bakery" projekt kom utifrån detta tanken, målet är att alla användaren skulle kunna beställa direkt från bageriets webbplats till bra priser, det ska vara enkel och effektiv.

3.2. Detaljerad problemformulering

Webbplatsen kommer att vara enkel att använda, det kommer finnas en sortiment sida som innehåller alla bakverk, bröd, sallader osv med allt information som behövs för att veta vad innehåller varan till exempel ingredienser och priser. Man kan lätt beställa igenom att gå till kontaktsida och använda kontaktformuläret eller igenom att ringa och beställa direkt.

Webbsidan ska göras av validerad kod och ska vara responsiv och passar till olika skärmstorlekar.
Projektet implementerad genom 2 veckor, från idén av projektets helhet och samling av material som till exempel bilder och texter till mockup design och implementeringen.

Svårigheter som dyk upp var mest med positionering med CSS och funktioner som ska genomföras med JavaScript.

Ett problem som tog ganska många försök och fundering var att kunna skapa en fast storlek på bildspelets kant, så att det inte ändrar med varje bild beroende på bildens höjd och bredd mått. Ett annat problem var att kunna spara kundens ärende där kundens skickar med kontaktformuläret och sen visa den på skärmen och allt med JavaScript samt validera formuläret. Leverans är väldigt viktigt information som måste vara med i webbplatsen, särskild i beställning sida. Det kan vara ett problem att webbsidan är fiktiv och bageriet finns inte i verksamheten och därför det kan vara lite svårt att skriva leverans detaljer.

4. Teori

Projektet är en webbplats för ett litet bageri som består av flera sidor och publicera på GitHub Pages. Här beskrivs tekniker som ska behövs för att förstå sten av rapporten.

JSON och Fetch, Sortimentet beskrivs filen menu.json, ett textbaserad dataformat som JavaScript kan läsa. Med fetch() görs en HTTP-förfrågan som returnerar ett Promise och svaret kan sen läsas som JSON. Ett vanligt mönster är att först hämta data vilket är sortimentmeny i detta projekt, tolka den och sen skapa en DOM-element som visas på skärmen.

Git, GitHub och GitHub Pages, Git är ett versionshanteringssystem som GitHub är en webbtjänst som lagrar Git-repon online. GitHub Pages är en tjänst som publicerar vår repo som innehåller sidor med HTML, CSS och JavaScript som en webbplats.

Utvecklingsverktyg- Koden skrevs i Visual Studio Code, webbläsarens utvecklarverktyg DevTools som används för felsökning, samt för att se filer som inte kunde laddas eller funktioner som inte funkar.

Media queries- media queries ändrar layouten beroende på visningsytans storlek och med en brytpunkt "breakepoint" där det börjar från mobilvyn och efter specifika enhetsmått som till exempel 786px ändrar layouten för att anpassa större skärmar.

Web Storage API, LocalStorage- localStorage är webblagring sätt som låter en webbplats att spara data i användarens webbläsare. Datan lagras med nyckel/värde-par i textform och kan rensas av användaren eller webbläsaren. Datan skrivs med **setItem()** och läses eller hämtas med **getItem().**

5. Metod

Sätta plan för projektet och skriva alla punkter och idéer som ska finnas i webbplatsen. Planera varje sida innehåll och till exempel vilka bilder, färger och text ska den ha, samt vad ska den ha för funktioner det vill säga vad kan användaren göra i den sida.

Weirframes och Mockup kommer användas för att ge en snabb och tydlig bild över färger och typsnitt i Figma verktyg. Mockup ska visar sidans layout och struktur och bilderna ska hämtas från Pexels.com.

Data kommer hämtas med hjälp av Fetch API från en JSON-fil som ligger i samma repo och som innehåller sortimentmeny. Det kommer visa kort som ska skapas och det ska ha varans bild och information.

Projektet ska byggas i Visual Studio Code och ska version hanteras med Git, GitHub och GitHub Pages för att publicera den färdiga webbplatsen och som hjälper oss att kunna se kodens utveckling under hela processen. All arbetet ska genomföra i branch som ska skapas och sen när det är färdig slår den ihop med main branch. Repon ska också innehålla en fil med Figmas designen och denna rapport filen. Det kommer även finnas en README fil som ska innehålla en kort presentation av projektet och den färdig länk till webbplatsen.

Dynamiska sidor ska skapas med JavaScript för interaktivitet och funktionalitet som passar webbplatsens innehåll, målgrupp och syftet. För layouten dvs alla designer ska skapas med CSS, det kommer användas många modeller för flexibelt och responsiv sida som anpassar för olika skärmstorlekar. Till exempel som Flexbox, Grid och media queries.

Sen ska alla koder från HTML och CSS sidor valideras i W3Cs HTML och CSS validator.

6. Konstruktion

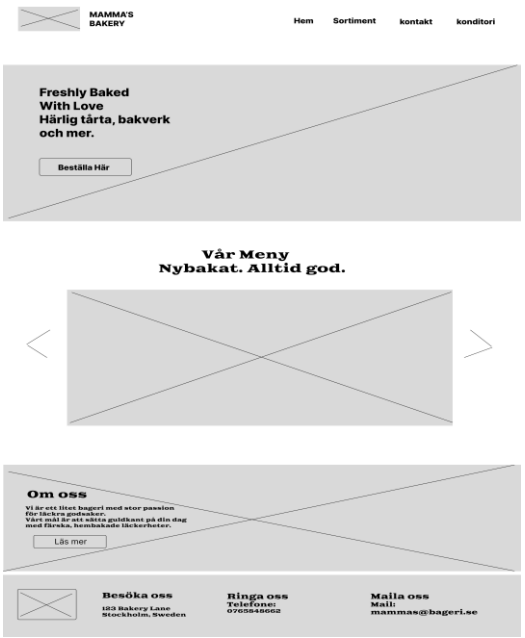
6.1 Planering och projekts struktur

Planerad först strukturen av webbplatsen sidor och hur ska de delas och vad behöver de innehållas. Webbplatsen består av fem sidor (huvudsida, sortiment, kontakt, order och about) som delar samma header och footer och filerna är sorterat i mappar efter typ, dvs till exempel alla CSS sidor i en CSS-fil och alla JavaScript sidor i en JS mapp.

Varje HTML sida ska ha sin eget CSS style sida och JavaScript sida, för att slippa en väldigt lång fil och undvika errors och på så sätt blir det lättare att hitta fel eller ändra något i style eller JS-kod och inte påverka andra kod. Versionshantering skedde med Git och publicerades med GitHub Pages. Alla utvecklingsarbetet gjordes i develop branch som skapades och sen när webbplatsen var färdig att publicera slog ihop branchen med main branch.

6.2 Grafisk profilen

I Figma två wireframes skisser skapades med grund idén och struktur en för mindre skärmar och en för större skärmar de presenterar i figur 1:1 och 1:2



Figur 1:1 – Wireframe 1



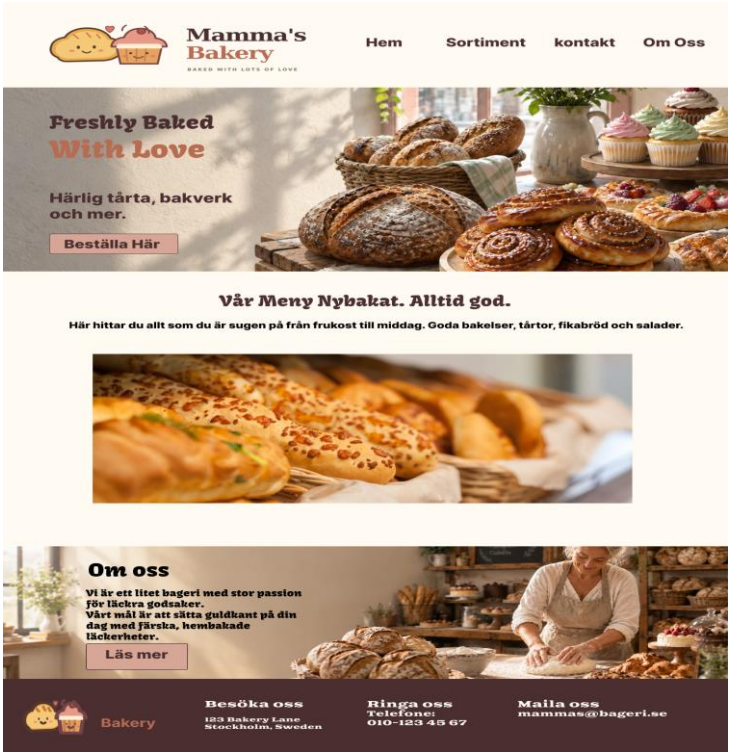
Figur 1:2-Wireframe 2

Tanken är att designen ska ge en varm och hemtrevlig känsla som passar ett litet bageri. Namnet på bageriet ge en trevlig känsla Mamma's Bakery och färgerna som mjuk rosa gräddvit och mörk maroon passade perfekt särskild att logotypen har samma färger. Även logotypen är anpassad för att ha den varm känslan med cupcake och bröd. Med de tankar och färger skapades

mockup till huvudsidan, här i figur 1:4 ser man logotypen och i figur 1:5 ser man mockup.



Figur 1: 3 – Logotyp



Figur 1:4- Mockup

Typsnittet är Chicle från Google fonts som ger sidan lekfull karaktär. Startsidan inleds med en hero-sektion med bild, rubrik och tydlig knapp för beställning sidan.

6.3 Responsive layout

Layouten är gjord med Mobile-first design med grundstyling för små skärmar först och med media-queries anpassar sig till större skärmar vid breakepoint 768px ändrar layouten till större skärmar med flexbox. Ett riktigt bra exempel är menyn som är dold på mobil och öppnas med hamburgerknapp "menyn knapp". På större skärmar döljs knappen och menyn visas i rad.

```css
.menu{display:none;width:100%;}
.menu.open{display:block;}
@media (min-width:768px){
    .menu-troggle{display:none;}
    .menu{display:block;width:auto;}
    .menu ul{display:flex;gap: 90px;}
```

Figur 1:5-Media queries

JavaScript ligger till och tar bort klassen open när man klickar på knappen med EventListener:

```javascript
//Skapar en eventlistener med funktion som öppnar och stänger menyn
menubtn.addEventListener("click", function(){
    menu.classList.toggle("open")
});
```

Figur 1:6-Menyn knapp

Lösningen valdes eftersom den bara kräver en klass som växlas och själva utseende ligger kvar i CSS. Det gjordes med en kort kod i JavaScript med EvenListener.

6.4 Automatiskt Bildspelet på startsidan

Med inspiration från W3school webbsidan skapades ett bildspel i startsidan ett bildgalleri som innehåller några av bageriets sortiment bilder med textbeskrivning av bilden. Den är byggd med semantiskt element, tex figure och figcaption.

```html
<!--Här är en section for slideShow med bilder-->
<section class="slideshow-container" aria-label="Vår meny i bilder">
    <figure class="mySlide">
        <div class="numbertext">1/7</div>
        <img src="images/bullar.jpg" alt="svensk-bullar" >
        <figcaption class="text">Bullar</figcaption>
    </figure>
```

Figur 1:7-Bildspelet

Alla bilder gömd som standard med (display: none) och JavaScript visar en i taget. Problemet som tog ett tag att lösa var att rutan ändrade storlek beroende på bildens mått och det löstes med CSS genom att ge figuren som innehåller bilder en fast höjd och låt bilder täcka ytan med objekt-fit: cover. Nu för JavaScript funktion som kör det spel

```
//SlideShow koder:
let slideIndex=1;
showSlides();

function showSlides(){

    let i;
    //Hämtar element från DOM
    const slides = document.getElementsByClassName("mySlide");
    //Loopar igenom alla bilder i div
    for(i =0; i < slides.length; i++){
        slides[i].style.display = "none";
    }

    slideIndex++;
    if(slideIndex > slides.length){slideIndex = 1}//Går tillbaka
    slides[slideIndex-1].style.display = "block";
    setTimeout(showSlides, 3000); //Ändrar bild varje 3 sekunder
```

Figur 1:8 Bildspel funktion i JavaScript

Med hjälp av for-loop loopar det igenom alla bilder och sen med If-satsen när alla bilder är slut så börjar den om och loopa från bild 1. SetTimeout gör att bilden ändrar varje 3 sekunder.

6.5 Sortiment med fetch och map()

Sortiment sida bygger sin produkt kort dynamiskt med JavaScript och html innehåller två div "message och menu-list" som ska fyllas sen med hjälp JavaScript funktioner. Alla produkter ligger i en json-fil och ska hämtas därifrån, varje produkt i filen är ett objekt med olika information såsom kategori, titel, pris, ingredienser osv. Detta sätt underlätta att ändra sortiment i json-fil utan att html ska påverkas och alla produktkort får samma struktur och styling. Här i figur 1:9 visas JavaScript koden som körs i sortimentsida, hämtning av data och skapande av produkt kort:

```
//Här skrivs sortiment kort lista som ska skrivas ut
//fetch anrop
fetch("./menu.json")
.then(response => {
    if(!response.ok) throw new Error ("Sortiment kunde inte laddas!");
    return response.json();
})

//Skapar kort till
.then(varor => {
    sortiment.innerHTML = varor.map(vara =>
        `<article class="kort">
        <img src="${vara.image}" alt="${vara.imageAlt}" loading="lazy">
        <div class="kort-innehall">
          <span class="kategori">${vara.category}</span>
          <h2>${vara.title}</h2>
          <p>${vara.description}</p>
          <p><strong>Ingredienser:</strong> ${vara.ingredients.join(", ")}</p>
          <p class="pris">${vara.priceSEK} kr</p>
        </div>
      </article>`).join("");
})

.catch(error => {
    document.getElementById("message").textContent =   "Något fel hänt, menyn
    console.error(error);
});
```

Figur 1:9 Sortiment hämtning och skapande av kort

Först hämtas filen med fetch med relativ sökväg, sen kontrollerar om fetch inte lyckas vid hämtning hastar det ett fel. Svaret den omvandlar till en array med response.json() och efter det omvandlar map() varje produktobjekt i json-fil till ett kort. Map() valdes här eftersom varje produkt ska ges här exakt ett kort och ursprungligt data ska inte ändras.

Ingredienser görs om till kommaseparerat text med join(,) och den färdiga arrayen av strängar slås ihop med join(""), utan det hamnar kommatecken mellan korten.

Loading: "lazy " användes för att webbläsaren ska vänta med att hämta bilder som ligger utanför skärmen alltså inte syns i skärmen än tills användaren scrollar och närmar sig. Det minskar mängder av data som behöver laddas när sidan öppnas.

För felhantering finns det två delar, den första är meddelande som skrivs i consolen för utvecklaren, medan användaren får meddelande i message elementmed rött text så att det syns.

CSS Grid används för alla kort på sortiment sida, på mindre skärmar som mobil visas det ett kort per rad medan på större skärmar som datorer bärbara datorer visar de i fyra kolumner. Grid här var den bästa alternativ för kort eftersom den ordnas innehåll i både rader och kolumner.

```css
/* Dator */
@media (min-width: 768px) {
  .kort-grid {
    grid-template-columns: repeat(4, 1fr);
    min-width: 1100px;
  }
}
```

Figur 1:10

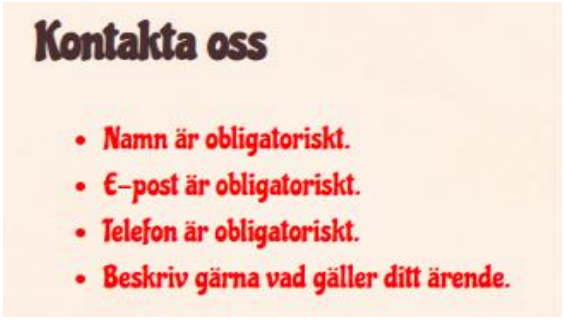
```css
/*Sortiment kort*/
.kort-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
  max-width: 270px;
  margin:auto;
  padding: 16px;
}
```

Figur 1:11

6.6 Kontaktformulär och localStorage lagring

Det finns två sektioner en med formuläret och en med bekräftelse-meddelande och som visar sparade historik som sparar i localStorage. Bekräftelse är dold från början med display: none i CSS och som visas på skärmen med JavaScript när ett ärende har sparats. På den set får användaren en bekräftelse att ärende har skickat och får se ärendet innehåll. I den enkel sida som skapat med bara HTML, CSS och JavaScript så kan vi inte än skicka ärendet verkligen eller göra något med ärendet som fylls i formuläret, men istället sparar vi historiken alltså ärendet i localStroage med hjälp av JavaScript funktioner.

Det finns en ul-lista som visar error message med rött text som en list ovanför kontaktformulär vid fel inmatning eller tomma fält. I HTML: et används fieldset, legend och label för att gruppera fälten och koppla for med id. Fälten har rätt typ vilket ger ett anpassat tangentbord för mobil.



Figur 1:12 Error lista för felmeddelande



JavaScript som körs formuläret är delat i några små funktioner för att vara tydlig och lätt att läsa och ändra. Först hämtas element med query-Selector och sparas med konstanter. Skapat en nyckel för att spara datan under (`const STORAGE_KEY = "kunduppdrag";`) i localStorage. I figur 1:13 Nyckeln och Arrays.

```
//Nyckeln som historiken sparas under i localStorage
const STORAGE_KEY = "kunduppdrag";

//Array som har sparade historik ärende (laddar tidigare ärende)
let history = JSON.parse(localStorage.getItem(STORAGE_KEY)) || []; //Hämtar data

// Array som används för felmeddelanden
let errors =[];
```

Figur 1:13 Nyckeln och Arrays

Raden med history är väldigt viktigt den läser in tidigare sparade ärende när sidan öppnas och getItem() hämtar data, returnerar null om arrayen är tomt alltså om inget ärende sparat. JSON.pars() omvandlar till JavaScript för att kunna läsa ut om datan finns och operatorn || [ ] ersätter det med en tomt array. Errors är en lista som töms och fylls vid varje validering.

Validerings problem löstes med två funktioner, det första är validateForm() som körs varje gång formuläret skickas. Börjar med att tömma errors lista sen läser forms fältens värde och tar bort mellanslag i början och slutet med trim(). För varje fel eller regeln som inte uppfylls lägger till ett meddelande i errors lista med hjälp också av if-satsen. Exempel på koden visas i figur 1:14

```
function validateForm(){
    errors = [];
    //Läs värdena från formuläret och trima bort mellanslag
    const fullname = fullnameInput.value.trim();
    const email = emailInput.value.trim();
    const phone = phoneInput.value.trim();
    const text = textArea.value.trim();

    //kontrollera obligatoriska fält
    if(fullname ===""){
        errors.push("Namn är obligatoriskt.");
    }
```

Figur 1:14 ValidateForm funktion

Alla fält Kontrolleras innan funktionen är klar så att besökaren ser alla fel på en gång och rätta alla istället för att rätta ett i taget. Textrutan har ett längdkrav på 20 teckan för att ärende ska innehålla tillräckligt med information. Funktionen slutar med (return error.length === 0) dvs true när allt är giltigt annars false och visar fel meddelande med funktionen visaErrors() som först rensar gamla list med (innerHTML ="") sen skapar en li element för varje felmeddelande med forEach.

```
//Visar felmeddelanden på sidan
function visaErrors(){
    //Rensa tidigare ressultat
    errorList.innerHTML = "";

    //Skiver ut felmeddelande
    errors.forEach(function (error){
        const li = document.createElement("li");
        li.textContent = error;
        errorList.appendChild(li);
    })
}
```

Figur 1:15 Error Funktion som skapar li element för errors

Med funktion creatQuestion skapar ett objekt med alla fälten och lägger det sist i arrayen med push och anropar saveInStorage() funktion som sparar ärende i localStorage med setItem() och JSON.stringify() som omvandlar tillbaka datan till JSON-sträng under en och samma nyckel.

För att visa bekräftelse funktionen showConfirmation() visar bekräftelsesektionen och hämtar det senaste ärende. Namn och ärendetext skrivs ut i ett stycke.

```
//Funktion som visar bekräftelse och historik
function showConfirmation(){
    confirmation.style.display = "block";
    //Tömmeer diven
    historySection.innerHTML = "";

    //Hämtar det sista ärende i arrayen
    const senaste = history[history.length - 1];

    //Skapar ett stycke med det senaste ärende
    const p = document.createElement("p");
    p.textContent = senaste.fullname + "-" + senaste.text;
    historySection.appendChild(p);

}
```

Figur 1:16 Visar Bekräftelsen

Sist så finns det en händelselyssnare som knyter ihop alla funktioner och styrs med "submit" lyssnare. Det innehåller också en event.preventDefault() som stoppar webbläsarens standard beteende, så att sidan inte ska laddas om. Form.reset() tömmer formuläret efter den skickas.

6.7 Gemensamma header och footer

Alla sidor har samma struktur och design för header och footer med samma CSS styling och footer Styling för att ge mer professionellt och snyggt utseende. Även alla sidor innehåler i deras JS-sidor samma kod för hamburgaremeny så att det stängs och öppnas på små skärmar och försvinner i större skärmar.

```
//Hämtar element från DOM
const menubtn = document.querySelector(".menu-troggle");
const menu = document.querySelector(".menu");

//Skapar en eventlistener med funktion som öppnar och stänger menyn
menubtn.addEventListener("click", function(){
    menu.classList.toggle("open")
});
```

Figur 1:17 Hamburgaremenyn

Bara beställning sida har inte JS-sidan utan script koden finns i HTML: et eftersom den inte innehåller mycket information utan bara en text och är onödigt att skapa en JS-sidan bara för två rader med kod.

```
<!--Här kommer JS-kod för meny-knapp-->
<script>
//Hämtar element från DOM
const menubtn = document.querySelector(".menu-troggle");
const menu = document.querySelector(".menu");
//Skapar en eventlistener med funktion som öppnar och stänger menyn
menubtn.addEventListener("click", function(){
menu.classList.toggle("open")
});
</script>
```

Figur 1:18 Script kod i beställnings HTML: et

Även CSS koder för header och footer finns samma i alla sidor.

6.8 Publicering och validering av sidor

Webbplatsen publicerade till GitHub med namnet "project*_Luccian-Ghazoul " i main-branch. Alla sidor är validerat med W3C validator utan något fel eller varningar. Webbplatsen är testat i olika skärmstorlekar och funkar bra.*

7. Resultat

Resultatet av projektet blev en färdig och responsiv webbplats för Mamma’s Bakery som publicerades med hjälp av GitHub Pages. Webbplatsen består av fem sidor: startsida, sortiment, kontakt, om oss och beställning. Sidorna har en gemensam design med samma färger, typsnitt, header och footer för att skapa en enhetlig känsla.

På startsidan kan besökaren få en introduktion till bageriet och se ett automatiskt bildspel med bilder från sortimentet. På sortimentsidan visas produkterna dynamiskt genom att informationen hämtas från en JSON-fil med hjälp av Fetch API. Sortimentet innehåller 32 produkter som är indelade i fyra kategorier: bröd, bakverk, desserter och sallader. Varje produkt visas med bild, namn, beskrivning, ingredienser och pris.

Kontaktsidan innehåller ett kontaktformulär där besökaren kan fylla i sina uppgifter och skriva ett meddelande. Formuläret kontrollerar att obligatoriska fält är ifyllda och att meddelandet innehåller minst 20 tecken. När formuläret fylls i korrekt sparas ärendet i webbläsarens LocalStorage och ett bekräftelsemeddelande visas för besökaren. Beställningssidan informerar om hur kunden kan beställa via telefon eller kontaktformuläret.

Webbplatsen testades på både mobil och dator. Resultatet visade att layouten anpassar sig till olika skärmstorlekar och att navigeringsmenyn fungerar på mindre skärmar genom hamburgermenyn. Även webbplatsens funktioner, såsom bildspelet, sortimentets hämtning och kontaktformuläret, testades.

HTML- och CSS-koden kontrollerades även med W3C-validator. Sammantaget blev resultatet en webbplats som uppfyller projektets huvudsakliga mål och som ger besökaren möjlighet att enkelt ta del av bageriets information och sortiment

8. Slutsatser

Jag är nöjd med resultatet eftersom webbplatsen uppfyller projektets huvudsakliga mål. Den innehåller fem sidor, har en enhetlig design och fungerar på både mobil och dator.

Under arbetet stötte jag på några problem, framför allt med CSS-positioneringen i bildspelet och JavaScript-funktionerna i kontaktformuläret. Genom att testa olika lösningar kunde jag lösa problemen. Jag lärde mig också hur viktigt det är att dela upp koden i mindre funktioner för att göra det enklare att förstå och felsöka.

En styrka med webbplatsen är den responsiva designen och att sortimentet hämtas dynamiskt från en JSON-fil. En svaghet är att kontaktformuläret endast sparar information i användarens webbläsare och inte skickar den till bageriet.

Om jag skulle utveckla projektet vidare skulle jag vilja skapa en riktig beställningsfunktion och koppla kontaktformuläret till en server. Jag skulle även vilja testa webbplatsen med fler användare för att få feedback.

En svaghet är att det inte finns någon leveranssektion med alla leverans detaljer och information, samt en funktion för leverans alternative där kunden kan välja och skriva adressen som varorna ska levereras till. Detta kan jag inte utveckla under projektet men det är en viktig förbättring som jag skulle välja arbeta vidare med.

Sammanfattningsvis har projektet gett mig mer kunskap om HTML, CSS, JavaScript och responsiv webbdesign. Jag är nöjd med slutresultatet och tycker att projektet har varit en bra möjlighet att använda mina kunskaper i praktiken.

Källförteckning

Källor:

[1] MDN Web Docs. Working With JSON. [Internet] https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/JSON (2026-10-22)

[2] W3school. How To Slideshow. [Internet]

https://www.w3schools.com/howto/howto_js_slideshow.asp (2026-10-22)

[3] MDN Web Docs. Using The Web Storage API. [Internet]

https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API/Using_the_Web_Storage_API (2026-10-23)

[4] MDN Web Docs. Media Queries fundamentals. [Internet]

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Media_queries (2026-10-22)

[5] GitHub Docs. What Is GitHub Pages?. [Internet]

https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages (2026-10-28)

[6] W3C. Markup Validation Service och CSS Validation Service [Internet]. Hämtad från: https://validator.w3.org/ (2026-10-30)

[7] Google Fonts. [Internet] https://fonts.google.com/ (2026-10-30)

[8] Canvas.miun.Moments föreläsningar.[Internet] https://canvas.miun.se/courses/11718 (2026-10-27)